

ПРАКТИКУМ - ПОЛНАЯ ПОШАГОВАЯ ВЕРСИЯ

Локальные AI-агенты, Git и DevOps

Windows + macOS | Codex + Hermes | Ollama + Open-LLM- VTuber

Windows | PowerShell | Open-LLM-VTuber 1.2.1 | Ollama | Codex CLI

Редакция: август 2026

Как работать. Открывайте эксперимент, копируйте готовый prompt целиком, выполняйте команды только в указанном окне и сверяйте реальный результат с разделом «Что должно получиться». Обращение за дополнительным разъяснением должно требоваться только при новой, неописанной ошибке.

Поле автора / преподавателя: _____

Часть I. Самостоятельная настройка с нуля

Назначение. Эта часть не требует предыдущего практикума. Она ведёт от чистой Windows- или macOS-системы до локального голосового и экранного агента, а затем показывает три взаимозаменяемых маршрута LLM: прямой Ollama, Hermes API Server или минимальный local-agent-gateway.

1. Что именно собирается

Итоговая система состоит из независимых компонентов. Их нельзя называть одним словом «агент»: у каждого своя роль.

Компонент	Роль	Где работает	Обязателен
Codex	Инженер: читает репозиторий, планирует, меняет файлы, запускает проверки	Desktop/CLI	Нет, если используется Hermes как инженер
Hermes Agent	Инженер и/или OpenAI-совместимый runtime с инструментами	CLI/Desktop/API server	Только для Hermes-маршрута
Open-LLM-VTuber	Интерфейс, Live2D, диалог, WebSocket, ASR/TTS orchestration	Python-сервер + браузер/Electron	Да
Ollama	Локально запускает LLM/VLM	127.0.0.1:11434	Да для локального маршрута
Sherpa-ONNX	Локально распознаёт микрофон	В процессе VTuber	Для voice baseline
pyttsx3	Локально озвучивает ответ системным голосом	В процессе VTuber	Для локального TTS
Local gateway	Bearer auth, allowlists, request ID, безопасные ошибки и streaming	127.0.0.1:8642	Один из gateway-маршрутов
OpenRouter	Необязательный облачный провайдер моделей	Интернет/API	Нет

Схема потоков

```
Микрофон -> Sherpa-ONNX ASR -----+
Скриншот/экран -> Open-LLM-VTuber -> выбранный LLM endpoint |
                                     v
                                     ответ -> pyttsx3 -> динамики

Маршрут А: Open-LLM-VTuber -> Ollama:11434
Маршрут В: Open-LLM-VTuber -> Hermes:8642 -> выбранная Hermes-модель
Маршрут С: Open-LLM-VTuber -> local-agent-gateway:8642 -> Ollama:11434
```

Главное различие. Codex не является постоянным локальным HTTP gateway. Он может создать, настроить и проверить gateway, но Open-LLM-VTuber не может отправлять runtime-запросы непосредственно в текущий чат Codex.

2. Как выбрать маршрут

Нужен результат	Выбор	Почему
Самый простой полностью локальный VTuber	Прямой Ollama	Меньше процессов и конфигурации
Hermes, память и инструменты внутри runtime	Hermes API Server	OpenAI-compatible API и полный toolset Hermes
Минимальный контролируемый шлюз к Ollama	local-agent-gateway	Узкий код, allowlists, тесты и нейтральные ошибки
Быстро менять облачные модели	Hermes + OpenRouter или прямой OpenRouter	Один API, но данные уходят во внешний сервис

Рекомендованный учебный маршрут. Сначала докажите прямой Ollama baseline. Затем добавьте ровно один gateway. Так ошибка всегда локализуется между уже проверенными границами.

3. Что человек всё равно делает сам

- Подтверждает установщики и системные диалоги Windows/macOS.
- Проходит вход в ChatGPT/Codex, GitHub, Hermes/Nous Portal или OpenRouter.
- Даёт браузеру доступ к микрофону, камере или демонстрации экрана.
- Произносит тестовую фразу и подтверждает, был ли слышен звук.
- Решает, публиковать ли ветку и создавать ли draft PR.
- Никогда не вставляет секрет в чат: вводит его локально в защищённый файл или системный prompt.

Почему AI не может сделать буквально всё. Авторизация, выбор устройства ввода, прослушивание динамиков и выдача GUI-разрешений являются границами человеческого контроля. Хороший агент доходит до такой границы, сообщает одну точную операцию и после неё продолжает без повторного объяснения.

4. Требования к компьютеру и каталогу

Параметр	Windows	macOS
ОС	Windows 10/11 x64; PowerShell 7 желательно	macOS 13+ желательно; Intel или Apple Silicon
Терминал	Windows Terminal + PowerShell 7	Terminal/iTerm + zsh
Рабочая папка	C:\dev\agent-lab	~/dev/agent-lab

Путь	ASCII, без OneDrive и длинных имён	Без iCloud Drive и сложных пробелов
RAM	16 ГБ минимум для 4B VLM; больше лучше	16 ГБ unified memory минимум; больше лучше
Свободное место	Не менее 15-25 ГБ	Не менее 15-25 ГБ

5. Предварительная установка

Устанавливайте системные компоненты вручную. AI сначала проверяет наличие и только затем предлагает отсутствующее. Не запускайте одновременно несколько package manager-команд.

Установка базовых инструментов

Windows PowerShell 7

```
winget install --id Microsoft.PowerShell --source winget
winget install --id Git.Git -e
winget install --id GitHub.cli -e
winget install --id Gyan.FFMpeg -e
winget install --id astral-sh.uv -e
# Ollama: установите официальное Windows-приложение или:
winget install --id Ollama.Ollama -e
# После установки закройте и заново откройте Terminal.
```

macOS Terminal (zsh)

```
# Установите Homebrew с brew.sh, если его ещё нет.
brew install git gh ffmpeg uv
brew install --cask ollama
# Запустите Ollama из Applications один раз.
exec zsh
```

Проверка

Windows PowerShell 7

```
pwsh --version
git --version
gh --version
uv --version
ffmpeg -version
ollama --version
gh auth status
```

macOS Terminal (zsh)

```
zsh --version
git --version
gh --version
uv --version
ffmpeg -version
ollama --version
gh auth status
```

Нормальный результат. Все version-команды завершаются с кодом 0. `gh auth status` может

требовать `gh auth login`; токен нельзя копировать в отчёт.

6. Установка и запуск Codex

Codex используется здесь как инженерный агент. Выбирайте Desktop-приложение, если важны изображения и удобные разрешения, либо CLI для воспроизводимого терминального процесса.

Первый запуск

Windows PowerShell 7

```
# Рекомендуется официальный Codex Desktop installer.
# Для CLI используйте официальный установщик, показанный приложением/документацией.
codex --version
Set-Location C:\dev\agent-lab
codex
```

macOS Terminal (zsh)

```
# Установите Codex Desktop для macOS либо официальный CLI.
codex --version
mkdir -p ~/dev/agent-lab
cd ~/dev/agent-lab
codex
```

Windows sandbox. Если CLI сообщает, что `codex-windows-sandbox-setup.exe` не найден, сначала обновите Codex и перезапустите терминал. Не обходите проблему постоянным полным разрешением. Для разовой read-only команды допустимо выбрать однократное выполнение вне сломанной sandbox только после проверки команды и cwd.

macOS permissions. Первый доступ Codex к каталогу Desktop/Documents, браузеру или автоматизации может вызвать системный диалог Privacy & Security. Разрешайте только нужный каталог или действие; Full Disk Access для практикума не требуется.

PROMPT: безопасная проверка Codex

Работай как инженер в режиме контролируемого агента на Windows.
Текущая рабочая папка: C:\dev\agent-lab. Не предполагай, что команды другой ОС работают здесь.

Перед изменением файлов:

1. Покажи карту релевантной части репозитория и входные точки.
2. Сформулируй одну атомарную цель.
3. Перечисли файлы чтения и изменения.
4. Назови риски, предположения и реальные команды проверки.
5. Сначала покажи план. Не меняй файлы до моего явного «одобряю».

Ограничения:

- не используй sudo; не выполняй `rm -rf`, `git reset --hard`, `git clean -fd` и `force push`;
- не изменяй `main` и `upstream` без отдельного явного разрешения;
- не читай и не печатай `.env`, токены, SSH-ключи, `cookies`, `keychain` и полный `conf.yaml`;
- не трогай файлы вне текущего проекта;
- не ставь глобальные пакеты без отдельного обоснования;
- не скрывай ошибки и не заменяй тест утверждением;

- после изменения покажи краткий diff, реальный минимальный тест и git status --short;
- если требуется расширение scope, остановись и запроси новый атомарный шаг.

Классифицируй ошибки как environment, configuration, dependency, source или test limitation.

Не задавай вопросов, ответ на которые можно безопасно получить read-only проверкой. Останавливайся только перед секретом, GUI-разрешением ОС, внешней публикацией или расширением scope.

Текущая задача: только инвентаризировать среду. Выполни Get-Location и версии git, gh, uv, ollama. Ничего не изменяй. Не печатай токены.

PROMPT: тот же шаг на macOS

Работай как инженер в режиме контролируемого агента на macOS.

Текущая рабочая папка: ~/dev/agent-lab. Не предполагай, что команды другой ОС работают здесь.

Перед изменением файлов:

1. Покажи карту релевантной части репозитория и входные точки.
2. Сформулируй одну атомарную цель.
3. Перечисли файлы чтения и изменения.
4. Назови риски, предположения и реальные команды проверки.
5. Сначала покажи план. Не меняй файлы до моего явного «одобряю».

Ограничения:

- не используй sudo; не выполняй rm -rf, git reset --hard, git clean -fd и force push;
- не изменяй main и upstream без отдельного явного разрешения;
- не читай и не печатай .env, токены, SSH-ключи, cookies, keychain и полный conf.yaml;
- не трогай файлы вне текущего проекта;
- не ставь глобальные пакеты без отдельного обоснования;
- не скрывай ошибки и не заменяй тест утверждением;
- после изменения покажи краткий diff, реальный минимальный тест и git status --short;
- если требуется расширение scope, остановись и запроси новый атомарный шаг.

Классифицируй ошибки как environment, configuration, dependency, source или test limitation.

Не задавай вопросов, ответ на которые можно безопасно получить read-only проверкой. Останавливайся только перед секретом, GUI-разрешением ОС, внешней публикацией или расширением scope.

Текущая задача: только инвентаризировать среду. Выполни pwd и версии git, gh, uv, ollama. Ничего не изменяй. Не читай Keychain.

7. Установка и настройка Hermes Agent

Hermes может играть две роли: инженерный CLI и постоянный OpenAI-compatible API Server. Не смешивайте их в одном объяснении: интерфейс `hermes chat` не является endpoint, а `hermes gateway` является.

Установка Hermes

Windows PowerShell 7

```
iex (irm https://hermes-agent.nousresearch.com/install.ps1)
# Откройте новый PowerShell
hermes --version
hermes doctor
hermes setup --portal
```

macOS Terminal (zsh)

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
exec zsh
hermes --version
hermes doctor
hermes setup --portal
```

Если Nous Portal не используется, выполните `hermes model` и интерактивно выберите OpenRouter, Ollama или другой провайдер. Ключ вводится только в локальный secret prompt/конфигурацию Hermes.

Разделение профилей

Windows PowerShell 7

```
hermes profile create olv-runtime
olv-runtime setup
olv-runtime tools
# Для разработки отдельно:
hermes profile create coder
coder setup
```

macOS Terminal (zsh)

```
hermes profile create olv-runtime
olv-runtime setup
olv-runtime tools
# Для разработки отдельно:
hermes profile create coder
coder setup
```

Ограничение toolset. API Server Hermes даёт доступ к инструментам агента. Для профиля `olv-runtime` отключите terminal, file, browser, code_execution, delegation и cronjob, если VTuber не должен управлять компьютером. Профиль разделяет состояние, но не является filesystem sandbox.

Для включения Hermes API Server задайте в конфигурации профиля: enabled=true, host=127.0.0.1, port=8642 и уникальный bearer key. Предпочитайте `hermes dashboard` или `hermes config`; не публикуйте `.env` профиля.

Схема параметров — не вставляйте реальный ключ в документ или чат

```
API_SERVER_ENABLED=true
API_SERVER_HOST=127.0.0.1
API_SERVER_PORT=8642
API_SERVER_KEY=<UNIQUE_LOCAL_KEY>
# CORS не задавайте, если браузер не вызывает Hermes напрямую.
```

Запуск Hermes API Server

Windows PowerShell 7

```
olv-runtime gateway
# В другом окне:
Invoke-RestMethod http://127.0.0.1:8642/health
```

macOS Terminal (zsh)

```

olv-runtime gateway
# В другом окне:
curl -fsS http://127.0.0.1:8642/health

```

PROMPT: настроить Hermes как ограниченный runtime

Проведи настройку отдельного Hermes-профиля `olv-runtime` как loopback-only OpenAI-compatible API Server. Сначала read-only проверь текущую версию Hermes и актуальный синтаксис через `hermes --help` и `hermes gateway --help`.

Не читай и не печатай существующие secret files. Не генерируй ключ в сообщении. Попроси меня один раз ввести новый локальный bearer key через безопасный локальный ввод.

Настрой host 127.0.0.1, port 8642, обязательную bearer-аутентификацию и минимальный toolset без terminal, file, browser, code_execution, delegation и cronjob.

Не ослабляй CORS. До изменения покажи точные параметры и план. После подтверждения запусти gateway, проверь /health, отрицательный запрос без auth и один обезличенный chat-completions smoke test. В отчёте покажи только статусы, request id и redacted key length.

8. OpenRouter: только если нужен облачный маршрут

- Создайте ключ на сайте OpenRouter и сохраните его только в локальном secret store выбранного клиента.
- Проверьте актуальный model id перед каждым экспериментом: бесплатные модели и лимиты меняются.
- Не отправляйте закрытый код, `.env`, голосовые записи и несокращённые скриншоты.
- В Hermes выберите OpenRouter через `hermes model`; не подставляйте OPENROUTER_API_KEY в командную строку.
- В Codex OpenRouter не требуется для работы самого Codex; это отдельный runtime для приложения.

Граница данных. Облачная модель получает содержимое prompt и переданные изображения. Локальный Ollama не отправляет inference-запрос в облако, но первоначальная загрузка модели требует сети.

9. Создание рабочей папки и получение Open-LLM-VTuber

Клонирование

Windows PowerShell 7

```

New-Item -ItemType Directory -Force C:\dev\agent-lab | Out-Null
Set-Location C:\dev\agent-lab
git clone --recursive https://github.com/Open-LLM-VTuber/Open-LLM-VTuber.git
Set-Location .\Open-LLM-VTuber
git submodule status
git status --short

```


macOS Terminal (zsh)

```
mkdir -p ~/dev/agent-lab
cd ~/dev/agent-lab
git clone --recursive https://github.com/Open-LLM-VTuber/Open-LLM-VTuber.git
cd Open-LLM-VTuber
git submodule status
git status --short
```

Не используйте Code → Download ZIP. Frontend является Git submodule. Обычный ZIP может дать `Detail Not Found` и лишить проект корректной Git-истории.

Зависимости

Windows PowerShell 7

```
uv sync
```

macOS Terminal (zsh)

```
uv sync
```

Первый запуск может скачать ASR-модели. Не классифицируйте длительное отсутствие вывода как зависание, пока процесс жив и сеть/диск активны.

10. Воспроизведение upstream baseline

Запуск без изменения source

Windows PowerShell 7

```
Set-Location C:\dev\agent-lab\Open-LLM-VTuber
uv sync
uv run run_server.py --verbose
```

macOS Terminal (zsh)

```
cd ~/dev/agent-lab/Open-LLM-VTuber
uv sync
uv run run_server.py --verbose
```

- Успех: сервер слушает `localhost:12393`, браузер получает HTTP 200 и source tree не изменён.
- Если `conf.yaml!` отсутствует, первый запуск обычно создаёт его; остановите сервер Ctrl+C и проверьте файл по имени, не печатая содержимое.
- Если uv cache недоступен только агенту, повторите ту же команду с одноразовым разрешением вне sandbox. Это environment, а не dependency.
- Если процесс жив, но порт не слушает, сохраните время ожидания и классифицируйте как environment до появления traceback.

11. Ollama и qwen3-vl:4b

Модель и health check

Windows PowerShell 7

```
ollama list
ollama pull qwen3-vl:4b
Invoke-RestMethod http://127.0.0.1:11434/api/tags | Select-Object -ExpandProperty models
```

macOS Terminal (zsh)

```
ollama list
ollama pull qwen3-vl:4b
curl -fsS http://127.0.0.1:11434/api/tags
```

Порт занят. На Windows Ollama tray app и на macOS Ollama.app уже могут слушать 11434. Не запускайте второй `ollama serve`, если health check работает.

Текстовый smoke test

Windows PowerShell 7

```
$body = @{model='qwen3-vl:4b'; messages=@(@{role='user';content='Ответь одним словом: готово'});
stream=$false} | ConvertTo-Json -Depth 5
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:11434/v1/chat/completions -ContentType
'application/json' -Body $body
```

macOS Terminal (zsh)

```
curl -sS http://127.0.0.1:11434/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{"model":"qwen3-vl:4b","messages":[{"role":"user","content":"Ответь одним словом: готово"}],"stream":false}'
```

12. Безопасное изменение conf.yaml

Версия важнее примера. Не заменяйте весь `conf.yaml` фрагментом из другого релиза. Сначала найдите существующие ключи и их реальные отступы, затем меняйте только значения.

Для прямого Ollama выберите существующий `ollama\_llm`. Для gateway выберите существующий `openai\_compatible\_llm`. В релизе 1.2.1 путь начинается с `character\_config`, а не `system\_config`.

Маршрут	llm_provider	base_url	model/key
Прямой Ollama	ollama_llm	http://127.0.0.1:11434	qwen3-vl:4b; key не нужен
Hermes	openai_compatible_llm	http://127.0.0.1:8642/v1	обычно hermes-agent + API_SERVER_KEY

local gateway	openai_compatible_llm	http://127.0.0.1:8642/v1	qwen3-vl:4b + GATEWAY_BEARER_TOKEN
---------------	-----------------------	--------------------------	---------------------------------------

Бэкап и синтаксическая проверка

Windows PowerShell 7

```
Copy-Item .\conf.yaml ".\conf.yaml.before-setup.${Get-Date -Format yyyyMMdd-HH:mm:ss}"
# Откройте файл в локальном редакторе, не выводите его в терминал.
uv run python -c "from pathlib import Path; import yaml;
yaml.safe_load(Path('conf.yaml').read_text(encoding='utf-8')); print('YAML_OK')"
```

macOS Terminal (zsh)

```
cp conf.yaml "conf.yaml.before-setup.${date +%Y%m%d-%H%M%S}"
# Откройте файл в локальном редакторе, не выводите его в терминал.
uv run python -c "from pathlib import Path; import yaml;
yaml.safe_load(Path('conf.yaml').read_text()); print('YAML_OK')"
```

Реальная ошибка, которую нужно узнавать. Если `basic_memory_agent` и `llm_provider` имеют одинаковый отступ, provider становится соседним ключом и Pydantic/YAML падает. Исправляйте только отступ конкретной строки, затем полностью разбирайте YAML.

PROMPT: точно настроить conf.yaml

Работай только с локальным игнорируемым conf.yaml Open-LLM-VTuber. Не печатай весь файл и секреты. Сначала разберись с реальной YAML-структурой программно и покажи существующие пути для `basic_memory_agent.llm_provider`, `openai_compatible_llm.base_url`, `llm_api_key` и `model`. Не полагайся на путь из моего текста, если схема отличается. Атомарная цель: настроить выбранный мной маршрут, изменив минимальное число scalar values. Создай timestamped backup. До изменения покажи безопасный redacted diff и дождись подтверждения. После изменения полностью распарсь YAML и проверь точные пути, показывая ключ как `<redacted>` и только его длину. Source, templates и dependencies не меняй.

13. Локальный русский ASR

Выберите существующий backend `sherpa_onnx_asr`. Модели и допустимые значения определяются текущей версией проекта; не придумывайте имена ключей.

- В браузере разрешите микрофон только для localhost:12393.
- Сначала используйте наушники либо выключите динамики, чтобы доказать отсутствие feedback loop.
- Произнесите короткую фразу с уникальным числом, например «Проверка локального распознавания 4827».
- В evidence сохраняйте transcript, но не сохраняйте исходное аудио без необходимости.
- Cloud ASR keys и endpoints должны отсутствовать в активном пути.

Критерий. Уникальное число и смысл русской фразы распознаны приемлемо; в логах нет cloud request. Ошибки выбора микрофона — environment, неверный backend — configuration.

14. Локальный TTS через pyttsx3

Список голосов зависит от ОС. На Windows в нашем baseline был `Microsoft Irina Desktop`. На macOS сначала установите русский системный голос в System Settings → Accessibility → Spoken Content/System Voice, затем выберите имя, реально выведенное pyttsx3.

Список голосов без персональных путей

Windows PowerShell 7

```
uv run python -c "import pyttsx3; e=pyttsx3.init(); [print(i, v.name, getattr(v,'languages',[])) for i,v in enumerate(e.getProperty('voices'),1)]"
```

macOS Terminal (zsh)

```
uv run python -c "import pyttsx3; e=pyttsx3.init(); [print(i, v.name, getattr(v,'languages',[])) for i,v in enumerate(e.getProperty('voices'),1)]"
```

Изолированный TTS smoke test

Windows PowerShell 7

```
uv run python -c "import pyttsx3; e=pyttsx3.init(); v=next(x for x in e.getProperty('voices') if x.name=='Microsoft Irina Desktop'); e.setProperty('voice',v.id); e.say('Проверка локального синтеза 4827'); e.runAndWait(); print('TTS_COMPLETED')"
```

macOS Terminal (zsh)

```
# Замените <RUSSIAN_VOICE_NAME> только именем из предыдущего вывода.
uv run python -c "import pyttsx3; e=pyttsx3.init(); v=next(x for x in e.getProperty('voices') if x.name=='<RUSSIAN_VOICE_NAME>'); e.setProperty('voice',v.id); e.say('Проверка локального синтеза 4827'); e.runAndWait(); print('TTS_COMPLETED')"
```

Неинтерактивная сессия. COMError на Windows или отсутствие слышимого звука в sandbox/SSH не доказывает поломку pyttsx3. Повторите тест в обычном пользовательском Terminal с доступом к аудиосессии. На macOS проверьте выбранный output device и системный голос.

15. Минимальный local-agent-gateway

Этот маршрут воспроизводит реализованный нами gateway: loopback-only FastAPI, bearer auth, backend/model allowlists, request ID, ограничение размера, нейтральные 4xx/502, safe logging и streaming passthrough.

Получение и тесты

Windows PowerShell 7

```
Set-Location C:\dev\agent-lab
git clone --branch agent/local-gateway https://github.com/aka-gst/local-agent-gateway.git
Set-Location .\local-agent-gateway
uv sync --extra test
Copy-Item .\env.example .\env
# Заполните .env локально, не печатая его.
uv run pytest -q
```

macOS Terminal (zsh)

```
cd ~/dev/agent-lab
git clone --branch agent/local-gateway https://github.com/aka-gst/local-agent-gateway.git
cd local-agent-gateway
uv sync --extra test
cp .env.example .env
# Заполните .env локально, не печатая его.
uv run pytest -q
```

Имена параметров .env — реальный token не выводить

```
GATEWAY_BEARER_TOKEN=<32 RANDOM BYTES, BASE64URL>
GATEWAY_ALLOWED_BACKENDS=ollama
GATEWAY_ALLOWED_MODELS=qwen3-v1:4b
GATEWAY_DEFAULT_BACKEND=ollama
GATEWAY_OLLAMA_BASE_URL=http://127.0.0.1:11434/v1
GATEWAY_UPSTREAM_TIMEOUT_SECONDS=120
GATEWAY_MAX_REQUEST_BYTES=1048576
```

Запуск gateway

Windows PowerShell 7

```
uv run local-agent-gateway
# Отдельное окно:
Invoke-RestMethod http://127.0.0.1:8642/health
```

macOS Terminal (zsh)

```
uv run local-agent-gateway
# Отдельное окно:
curl -fsS http://127.0.0.1:8642/health
```

Почему timeout 120. При первом запросе qwen3-v1:4b может загружаться в память дольше 30 секунд. Сначала измерьте прямой Ollama-вызов; только затем увеличивайте timeout. `upstream unavailable` при рабочем Ollama часто является timeout, а не ошибкой модели.

Streaming. Open-LLM-VTuber фактически отправляет `stream=true`. Gateway должен передавать поток без накопления всего ответа, не пересылать клиентский Authorization в Ollama и закрывать upstream при disconnect.

16. Порядок запуска четырёх окон

Окно	Каталог	Процесс	Проверка
1	любой	Ollama.app/tray или ollama serve	11434 /api/tags
2	local-agent-gateway или Hermes profile	gateway	8642 /health
3	Open-LLM-VTuber	uv run run_server.py --verbose	12393 HTTP 200
4	любой	health/smoke commands	статусы и request id

Проверка портов

Windows PowerShell 7

```
Get-NetTCPConnection -State Listen -LocalPort 11434,8642,12393 -ErrorAction SilentlyContinue | Select-Object LocalAddress,LocalPort,OwningProcess
```

macOS Terminal (zsh)

```
lsof -nP -iTCP:11434 -iTCP:8642 -iTCP:12393 -sTCP:LISTEN
```

17. End-to-end smoke tests

1. Text-only: в VTuber отправьте «Ответь одним словом: готово». Должен появиться ответ, gateway — HTTP 200/request id.
2. Voice: включите microphone capture и произнесите фразу с новым уникальным числом. Сохраните только transcript и PASS/FAIL.
3. TTS: после одного ответа подтвердите слухом локальное воспроизведение. Затем выключите микрофон.
4. Loop control: на 15 секунд включите микрофон при динамиках и убедитесь, что новых бесконечных циклов нет; при риске используйте наушники.
5. Vision: используйте штатную кнопку camera/screen share или file input. Если ``document.querySelectorAll("input[type=file]").length`` равно 0, не имитируйте upload: отметьте capability NOT RUN.

Чего не доказывает UI-ответ. Сам ответ не доказывает маршрут. Нужны request id/HTTP 200 gateway и лог выбранного upstream без prompt/response содержимого.

18. Остановка и откат

6. Ctrl+C в окне Open-LLM-VTuber; дождитесь shutdown complete.
7. Ctrl+C в окне gateway; проверьте освобождение 8642.
8. Ollama tray/app можно оставить работающим; `ollama serve` остановите Ctrl+C.
9. Восстанавливайте conf.yaml только из известного timestamped backup при остановленном VTuber.
10. Не завершайте неизвестный PID. Сначала определите приложение-владельца порта.

19. Диагностика по симптомам

Симптом	Класс	Минимальная проверка	Действие
uv cache Access denied	environment	uv cache dir; повтор той же команды вне sandbox	Однократное разрешение или локальный cache внутри workspace
11434 address in use	environment	listener + /api/tags	Не запускать второй Ollama
YAML ParserError	configuration	полный safe_load + строки вокруг ошибки	Исправить только отступ/скаляя
Pydantic literal error	configuration	список допустимых backend из ошибки/кода	Использовать точный идентификатор
Gateway 401	configuration	сравнение token match без вывода	Одинаковый token и restart
Gateway 400 model not allowed	configuration	ollama list + allowlist	Точный model tag
Gateway 502 unavailable	environment/config	прямой Ollama + время	Запустить Ollama или увеличить timeout обоснованно
Chat endpoint error	configuration	base_url, health, model, request id	Исправить первую упавшую границу
pyttsx3 COMError	environment	интерактивный пользовательский терминал	Повторить вне sandbox
Нет file input	capability	DOM count и UI controls	NOT RUN; использовать camera/screen share
Звук не слышен	environment/config	системный output + изолированный TTS	Выбрать голос/output; не менять source
Feedback loop	configuration/environment	VAD, mic, 15 секунд	Наушники; настройка VAD, не повышать громкость

20. Один master prompt для AI

Этот prompt рассчитан на уже установленного Codex или Hermes. Он не отменяет системные диалоги и подтверждения внешних публикаций, но минимизирует уточняющие вопросы.

MASTER PROMPT — замените только ОС и маршрут

Я хочу с нуля настроить локальную связку Open-LLM-VTuber + Ollama qwen3-vl:4b + sherpa_onnx_asr + pyttsx3_tts на <Windows|macOS>.
LLM-маршрут: <прямой Ollama|Hermes API Server|local-agent-gateway>.

Работай как контролируемый инженер. Самостоятельно выполняй все безопасные read-only проверки и не задавай вопросы, ответ на которые виден в документации, --help, репозитории или локальном состоянии. Перед каждой группой изменений покажи атомарную цель, точные файлы, redacted diff, риски и команды теста; жди моего «одобряю». После одобрения заверши группу целиком.

Не используй sudo, destructive Git, force push, изменение main/upstream, глобальные пакеты без обоснования. Не читай и не печатай .env, conf.yaml целиком, token, SSH, cookies или keychain. Секрет запрашивай только через

безопасный локальный ввод и показывай лишь match/length. Не меняй source Open-LLM-VTuber, templates или dependencies, пока не доказан source defect и я отдельно не расширил scope.

Порядок:

1. Проверь ОС, shell, cwd, git, gh, uv, ffmpeg, ollama и выбранного агента. Классифицируй проблемы.
2. Получи Open-LLM-VTuber recursive clone в коротком несинхронизируемом пути; прочитай README/CLAUDE/pyproject/quick-start; покажи карту.
3. Выполни upstream baseline: uv sync, verbose server, HTTP 200, shutdown, source-tree unchanged.
4. Проверь Ollama, скачай qwen3-vl:4b только после подтверждения, выполни text-only smoke.
5. Создай backup conf.yaml и настрой минимальные реальные YAML paths выбранного маршрута; не заменяй файл шаблоном. Полностью распарсь YAML.
6. Настрой sherpa_onnx_asr. Получи список pytsx3 voices без путей; выбери доступный русский системный голос, на Windows предпочти Microsoft Irina Desktop, на macOS используй фактически установленный русский голос.
7. Если выбран gateway: настрой loopback 127.0.0.1:8642, bearer auth, allowlists, request id, безопасные ошибки и streaming. Для Hermes отключи опасные toolsets runtime-профиля. Для local-agent-gateway запусти pytest.
8. Запусти компоненты в правильном порядке и выполни health checks.
9. Остановись для моих ручных действий только четыре раза: browser login, secret input, microphone/screen permission и подтверждение слышимого звука.
10. Проведи text, voice, TTS, 15-second loop и доступный vision smoke. Не выдавай NOT RUN за PASS.
11. Покажи sanitised evidence: версии, backend names, request id/status, transcript тестовой фразы, TTS user-confirmed, loop result, source-tree status. Не показывай полный prompt/response.
12. Останови сервисы в обратном порядке и предложи ровно один следующий шаг.

Если команда одной ОС не подходит, сначала переведи её в native PowerShell 7 или zsh. Bash heredoc `<<'PY'` не используй в PowerShell. При любой ошибке покажи реальный вывод и не продолжай к следующей границе, пока текущая не доказана.

21. Формат итогового отчёта

Шаблон evidence

```
ОС: Windows / macOS
Маршрут: direct Ollama / Hermes / local-agent-gateway
Ollama model: <exact tag>
ASR: sherpa_onnx_asr — PASS / FAIL / NOT PROVEN
TTS: pytsx3_tts, voice=<name> — PASS / FAIL / NOT PROVEN
Gateway: HTTP status=<code>, request_id=<sanitized>
Vision: PASS / FAIL / NOT RUN; method=<screen/camera/file>
Loop control: PASS / FAIL / NOT RUN; observation=15 s
Source tree: unchanged / known diff
Secrets printed: no
Не доказано: <ограничения>
Следующий атомарный шаг: <ровно один>
```

22. Официальные источники и проверка актуальности

- Codex: <https://developers.openai.com/codex/>
- Codex Windows sandbox: <https://developers.openai.com/codex/windows/>
- Hermes Agent — установка, Portal и CLI: <https://hermes-agent.nousresearch.com/docs/>
- Hermes API Server: <https://hermes-agent.nousresearch.com/docs/user-guide/features/api-server/>
- Ollama — загрузка для Windows и macOS: <https://ollama.com/download>
- Ollama API: <https://docs.ollama.com/api>
- Open-LLM-VTuber Quick Start: <https://docs.llmvtuber.com/en/docs/quick-start/>
- Open-LLM-VTuber LLM configuration: <https://docs.llmvtuber.com/en/docs/user-guide/backend/llm/>
- OpenRouter API: <https://openrouter.ai/docs/api-reference/overview>
- uv: <https://docs.astral.sh/uv/>

- GitHub CLI: <https://cli.github.com/manual/>

Правило версий. Команды этого руководства сверены с локальным Open-LLM-VTuber 1.2.1 и окружением практикума. Если точный YAML path или CLI-флаг отличается, сначала прочитайте локальные README/CLAUDE/pyproject и --help, затем меняйте только фактически существующий ключ.

Часть II. Лабораторный практикум

Ниже сохранён подробный Codex-практикум V2: 15 экспериментов, отдельные prompts, команды, ошибки и критерии. Используйте его после Части I. Hermes можно подставить как инженерного агента, сохраняя тот же controlled-agent contract; в экспериментах 9-12 отдельно выбирайте Hermes API Server или минимальный gateway, а не оба одновременно.

macOS. Исходные лабораторные команды преимущественно даны для PowerShell. Перед выполнением на macOS используйте таблицу эквивалентов ниже и native zsh-команды из Части I. Не вставляйте PowerShell variables ``$env:`` и here-strings ``@'...'@`` в zsh.

Задача	Windows PowerShell	macOS zsh
Перейти в каталог	Set-Location PATH	cd PATH
Создать каталог	New-Item -ItemType Directory	mkdir -p
Скопировать файл	Copy-Item	cp
Проверить файл	Test-Path	test -f
Найти listener	Get-NetTCPConnection	lsof -nP -iTCP:PORT -sTCP:LISTEN
HTTP GET	Invoke-RestMethod	curl -fsS
Environment	<code>\$env:NAME='value'</code>	<code>export NAME='value'</code>
Активировать venv	<code>.\.venv\Scripts\Activate.ps1</code>	<code>source .venv/bin/activate</code>
Остановить foreground	Ctrl+C	Ctrl+C
Многострочный Python	<code>@' ... '@ uv run python -</code>	<code>uv run python - <<'PY' ... PY</code>

Как пользоваться этой версией

Эта редакция намеренно подробнее сокращённого практикума. Она рассчитана на последовательное выполнение: теория объясняет смысл шага, готовый prompt задаёт Codex безопасный score, PowerShell-блок даёт точные команды, а раздел проверки помогает не принять красивый ответ модели за доказательство.

Главное правило

Не импровизируйте со score. Если Codex обнаружил проблему, которой нет в текущей атомарной цели, зафиксируйте её и остановитесь. Не разрешайте агенту одновременно диагностировать, менять конфигурацию, патчить source и публиковать изменения.

Обозначения

Обычный PowerShell	Команда выполняется пользователем вне сессии Codex. Используется для интерактивного входа, UAC, foreground-сервисов и ручных проверок.
Codex CLI	Готовый prompt вставляется в новую или текущую сессию Codex из указанной рабочей папки.
После подтверждения	Не выполняйте следующие команды до того, как Codex покажет план и вы явно ответите «одобряю».
PASS	Есть реальный вывод, файл, test result или наблюдение, прямо доказывающее критерий.
NOT PROVEN	Утверждение возможно верно, но доказательства нет. Это не PASS.

Можно ли продолжить после эксперимента 8

Да. Не начинайте заново автоматически. Выполните эксперимент 8.5, сопоставьте старые результаты с критериями и повторите только проверки без доказательств. Если раньше ради pytsx3 менялся source, функциональный результат можно сохранить, но критерий source tree unchanged остаётся невыполненным до отдельного решения.

Роли компонентов

В оригинальной архитектуре Hermes совмещал инженерную роль и runtime gateway. В этой редакции обязанности разделены. Такое разделение важно: Codex не становится HTTP-сервисом только потому, что умеет писать и запускать код.

Codex	Анализирует репозиторий, предлагает план, меняет согласованные файлы, запускает тесты и работает с Git. Не является постоянным endpoint для VTuber.
Local gateway	Постоянно слушает 127.0.0.1:8642, принимает OpenAI-compatible запросы, проверяет Bearer token и маршрутизирует только разрешённые backend/model.
Ollama	Локально обслуживает qwen3-vl:4b через 127.0.0.1:11434/v1. Это основной offline backend.
OpenRouter/OpenAI	Опциональные cloud backends. В базовых voice/vision экспериментах выключены и не получают аудио.
Open-LLM-VTuber	Клиентская/runtime-система с WebSocket, Live2D, LLM, ASR и TTS. Подключается к gateway через local conf.yaml.

Поток данных

```
Codex -> читает/меняет/тестирует -> Git workspace
Browser microphone -> sherpa_onnx_asr -> text
text + screenshot -> Open-LLM-VTuber -> Local Gateway :8642
Local Gateway -> Ollama :11434/v1 -> qwen3-vl:4b
model response -> pyttsx3_tts -> Windows speakers
```

Приватность. ASR и TTS остаются локальными. Gateway не должен логировать полный prompt, response, Authorization и речь пользователя.

Рабочая структура и окна

Рекомендуемая структура

```
C:\dev\agent-lab\
|-- Open-LLM-VTuber\      # upstream exercise repository
|-- local-agent-gateway\  # отдельный gateway project
|-- evidence\            # только обезличенные доказательства
|-- praktikum_codex_gateway_ru_v2.docx
`-- praktikum_codex_gateway_ru_v2.pdf
```

Какие окна будут использоваться

1. PowerShell A: ollama serve. Окно остаётся открытым, пока работает локальная модель.
2. PowerShell B: gateway на 127.0.0.1:8642. Появляется с эксперимента 9.
3. PowerShell C: uv run run_server.py --verbose в Open-LLM-VTuber.
4. Codex CLI: отдельная сессия для анализа и согласованных изменений.
5. Browser: локальная страница VTuber, screenshot и microphone smoke tests.

Как понять, где вы находитесь

```
Get-Location
git branch --show-current
git status --short
```

Перед копированием команды. Сверьте путь в prompt с текущим Get-Location. Большинство ошибочных результатов в первых экспериментах возникало из-за запуска rg из C:\dev\agent-lab вместо корня Open-LLM-VTuber.

PowerShell: команды, которые часто путают

Bash heredoc не работает в PowerShell

Конструкция `python - <<'PY'` предназначена для Bash. В Windows PowerShell используйте `here-string`, затем передайте его в `stdin Python`.

```
@'
print("Проверка PowerShell here-string")
'@ | uv run python -
```

Остановка foreground-сервера

Нажмите `Ctrl+C` в том же окне, где запущен сервер. Закрытие окна допустимо как аварийный вариант, но в доказательстве лучше показать штатную остановку и отсутствие `listening port`.

Просмотрщик Codex

Если вывод открылся на полном экране и внизу написано `q to quit`, нажмите `q`. Это закрывает просмотрщик результата, а не удаляет задачу и не останавливает систему.

Красная строка `Failed to paste image`

Она означает, что CLI получил команду вставки изображения, но в буфере не было поддерживаемого `image format`. К `sandbox` и выполнению текстовой команды это не относится.

Никогда не публикуйте

- реальное значение `Token` из `gh auth status`;
- полный `conf.yaml` с ключами;
- пути `C:\Users\<имя>\...`;
- исходное аудио или `transcript` личной речи;
- скриншоты с чатами, `cookies`, `email` и ключами.

Универсальный controlled-agent контракт

Этот блок уже встроен в prompts изменяющих экспериментов. Если вы формулируете дополнительный атомарный шаг самостоятельно, начинайте с него.

Скопировать целиком

```
Работай как инженер в режиме контролируемого агента.
Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и
изменения, риски и команды проверки.
Сначала покажи план. Не меняй файлы до моего явного подтверждения.
Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
Не переключай и не изменяй main.
Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
Не трогай файлы за пределами указанного scope.
Не скрывай ошибки и не подменяй тест словесным утверждением.
После изменения покажи краткий diff, реальный минимальный тест и git status --short.
Если нужен больший scope, остановись и запроси новый атомарный шаг.
```

Как отвечать на запрос разрешения

- Yes, proceed - разрешение только текущей команды. Это безопасный выбор по умолчанию после проверки команды.
- Yes, and don't ask again - постоянное правило для префикса. Выбирайте только для узкого и понятного read-only префикса.
- No / Esc - команда отменяется. Используйте, если score шире задания, команда читает секреты или меняет неизвестные файлы.

Важно. Разрешение выйти за sandbox не делает опасную команду безопасной. Сначала проверяют точный target и причину escalation.

Стандарт доказательств

Практикум считается завершённым не тогда, когда агент написал «готово», а когда каждый критерий связан с фактом. Доказательство должно быть минимальным: достаточно нужных строк лога, а не полного дампа с персональными данными.

Команда	Точный текст команды, рабочая папка и exit code.
Результат	Минимальные строки, доказывающие ожидаемое состояние.
Ручное наблюдение	Допустимо для слышимого TTS и отсутствия feedback loop; явно помечается как ручное.
Git	git diff для изменяющего шага и git status --short после него.
Ограничение	Что тест не проверил. Например, health 200 не доказывает генерацию модели.

Шаблон записи результата

```

Эксперимент: <номер и название>
Дата/версия: <дата, commit или версия>
Команда: <точная команда>
Exit code: <0 или фактический код>
Доказательство: <минимальный обезличенный вывод>
Статус: PASS / FAIL / NOT PROVEN
Что доказано: <одно предложение>
Что не доказано: <одно предложение>
Git status --short: <пусто или фактические строки>
Следующий шаг: <только один>

```


Эксперимент 1. Инвентаризация среды

Атомарная цель. Зафиксировать исходное состояние Windows, инструментов, локальных моделей и Git-репозитория без изменения файлов.

Что мы делаем и зачем

Инвентаризация нужна как точка отсчёта. Если дальнейший запуск упадёт, вы сможете отличить исходную проблему окружения от ошибки, появившейся после настройки. Здесь не исправляют найденные проблемы внутри того же шага: сначала сохраняют факты и классифицируют их.

Что должно быть готово

- Codex CLI запускается из PowerShell.
- Рабочая папка C:\dev\agent-lab существует.
- Не требуется запускать Open-LLM-VTuber или менять conf.yaml.

Где выполнять

Обычный PowerShell: C:\dev\agent-lab. Затем новая сессия Codex из этой же папки.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 1: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Проведи read-only инвентаризацию среды C:\dev\agent-lab.

Ничего не изменяй. Не читай и не выводи содержимое .env, токенов, SSH-ключей, cookies, keychain и других секретов. Не печатай персональные абсолютные пути; заменяй каталог пользователя на %USERPROFILE% или %LOCALAPPDATA%.

Проверь реальные версии Git и uv, состояние GitHub CLI без значения token и scopes, список Ollama-моделей, найденные Git-репозитории и git status --short каждого из них. Если команда завершилась ошибкой, покажи exit code и классифицируй ошибку как environment, configuration, dependency или source.

Ничего не исправляй в этом шаге. В конце создай компактную таблицу: проверка, фактический результат, PASS/FAIL, классификация. Подтверди, что файлы не изменялись.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
Set-Location C:\dev\agent-lab
git --version
uv --version
gh auth status
ollama list
Get-ChildItem -Directory -Force | Where-Object { Test-Path (Join-Path $_.FullName '.git') }
git -C .\Open-LLM-VTuber status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 1: проверка результата

Что должно получиться

- Git и uv печатают версии и exit code 0.
- gh auth status сообщает успешный вход, но в отчёт не копируется строка Token.
- ollama list показывает qwen3-vl:4b или честно фиксирует отсутствие модели.
- git status --short не печатает строк для чистого рабочего дерева.

Типовые ошибки и действия

- orchestrator_helper_launch_failed: обновить Codex CLI и повторить минимальный Get-Location.
- Invalid GitHub token: выполнить gh auth login в обычном PowerShell и повторить только проверку.
- Ollama не может писать лог из sandbox: запустить ollama serve вне sandbox и разрешить агенту только ollama list.

Артефакты и доказательства

- [] Таблица версий без персональных путей.
- [] Список локальных моделей.
- [] Состояние Git-репозитория.
- [] Классификация всех ошибок.

Критерий завершения

- [] Каждая команда имеет реальный вывод.
- [] Source tree не изменён.
- [] В отчёте нет token и персональных абсолютных путей.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 1: Инвентаризация среды | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Если GitHub и Ollama исправны, перейти к безопасной подготовке fork и рабочей ветки.

Эксперимент 2. Fork, remotes и рабочая ветка

Атомарная цель. Убедиться, что работа ведётся в отдельной ветке собственного fork, а upstream main не будет затронут.

Что мы делаем и зачем

Ошибки remotes и target branch опаснее ошибки кода: корректный diff, отправленный в чужой upstream или main, нарушает правила практикума. Поэтому сначала проверяют URL, текущую ветку и чистоту дерева, и только затем создают учебную ветку.

Что должно быть готово

- Эксперимент 1 завершён.
- gh auth status успешен.
- Open-LLM-VTuber уже клонирован в C:\dev\agent-lab.

Где выполнять

C:\dev\agent-lab\Open-LLM-VTuber. Сначала read-only аудит в Codex; создание ветки - отдельное подтверждённое изменение.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 2: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Работай как инженер в режиме контролируемого агента.
 Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и изменения, риски и команды проверки.
 Сначала покажи план. Не меняй файлы до моего явного подтверждения.
 Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
 Не переключай и не изменяй main.
 Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
 Не трогай файлы за пределами указанного score.
 Не скрывай ошибки и не подменяй тест словесным утверждением.
 После изменения покажи краткий diff, реальный минимальный тест и git status --short.
 Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: проверить remotes, текущую ветку и статус Open-LLM-VTuber. Определи, какой remote является моим fork, а какой upstream. Не выполняй fetch, pull, push и switch до подтверждения.

Если текущая ветка main или master и дерево чистое, предложи создать lab/codex-practicum. Сначала покажи точную команду и попроси подтверждение. После подтверждения создай только ветку, затем покажи git branch --show-current и git status --short.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
Set-Location C:\dev\agent-lab\Open-LLM-VTuber
git remote -v
git branch --show-current
git status --short
# Только после подтверждения:
git switch -c lab/codex-practicum
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажимите Esc и объясните ограничение.

Эксперимент 2: проверка результата

Что должно получиться

- origin указывает на репозиторий пользователя.
- upstream, если существует, указывает на исходный Open-LLM-VTuber.
- Текущая ветка lab/codex-practicum.
- Рабочее дерево остаётся чистым.

Типовые ошибки и действия

- Ветка уже существует: не удалять её; выбрать другое имя только после проверки.
- origin указывает на upstream: остановиться и исправлять remote отдельным атомарным шагом.
- Есть неизвестные изменения: не переключать ветку до их классификации.

Артефакты и доказательства

[] Вывод `git remote -v` без credential.

[] Имя рабочей ветки.

[] `git status --short` после создания ветки.

Критерий завершения

[] main/master не изменён.

[] Нет push и force push.

[] Изменение ограничено созданием одной локальной ветки.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 2: Fork, remotes и рабочая ветка | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Перейти к read-only карте репозитория.

Эксперимент 3. Карта репозитория с Codex

Атомарная цель. Получить доказанную карту входных точек, конфигурации, модулей и тестов без записи.

Что мы делаем и зачем

Карта репозитория отвечает на практические вопросы: откуда запускается сервер, где валидируется `conf.yaml`, где создаются LLM/ASR/TTS и какие файлы относятся к upstream baseline. Без этой карты легко исправить симптом в неправильном слое.

Что должно быть готово

- Рабочая ветка подготовлена.
- Репозиторий открыт именно из его корня.
- Старые evidence и отчёты не нужны для анализа.

Где выполнять

Новая сессия Codex из `C:\dev\agent-lab\Open-LLM-VTuber`.

Scope

Разрешено

Только действия, прямо указанные в prompt и командах этого эксперимента.

Запрещено

Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.

Результат

Артефакты и критерии на третьей странице эксперимента.

Эксперимент 3: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Проведи read-only архитектурный обзор Open-LLM-VTuber.

Ничего не изменяй и не создавай. Не читай .env, evidence, .git, .venv, node_modules, старые логи и backup-файлы. Сначала прочитай README.md, CLAUDE.md или AGENTS.md, pyproject.toml, config_templates/README.md и quick-start документы, если они есть.

Покажи краткое дерево только релевантных каталогов. Затем укажи: серверную точку входа; команду запуска; конфигурационный слой; пользовательский conf.yaml; default templates; фабрики LLM, ASR и TTS; service context; WebSocket routing; frontend; Live2D; тесты и quality commands.

Каждое утверждение подтверди файлом и номером строки. Отдельно перечисли файлы, которые нельзя менять при upstream baseline. В конце выполни git status --short и объясни пустой вывод.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
rg --files -g README.md -g CLAUDE.md -g AGENTS.md -g pyproject.toml -g "*quick*" -g "!evidence/**"
rg -n "run_server.py|conf.yaml|tts_config|asr_config|llm_provider" README.md CLAUDE.md pyproject.toml
src config_templates
git status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 3: проверка результата

Что должно получиться

- Точка входа: `run_server.py`.
- Verbose-команда: `uv run run_server.py --verbose`.
- Схемы: `src/open_llm_vtuber/config_manager/`.
- Frontend отмечен как Git submodule, если это подтверждено локально.

Типовые ошибки и действия

- `rg` захватил `evidence`: повторить с `-g '!evidence/**'`.
- Номер строки не подтверждает тезис: открыть узкий диапазон и перепроверить.
- Агент предлагает менять код: отклонить, так как задача `read-only`.

Артефакты и доказательства

[] Карта каталогов.

[] Список входных точек с `file:line`.

[] Список запрещённых для `baseline` файлов.

Критерий завершения

[] У каждого принятого утверждения есть локальное доказательство.

[] `git status --short` пуст.

[] Ни один файл не создан.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 3: Карта репозитория с Codex | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Открыть новую сессию для независимого аудита, не передавая ей первый отчёт.

Эксперимент 4. Независимый аудит Codex

Атомарная цель. Получить вторую архитектурную карту другой формулировкой и проверить расхождения командами.

Что мы делаем и зачем

Второй аудит снижает риск якорения на первом ответе. Важно не просить модель подтвердить уже готовую карту, а сформулировать задачу независимо. Совпадение двух ответов ещё не доказательство; финальным доказательством остаётся локальный файл или команда.

Что должно быть готово

- Отчёт эксперимента 3 сохранён вне контекста новой сессии.
- Рабочее дерево чистое.

Где выполнять

Новая сессия Codex из корня Open-LLM-VTuber. Не продолжать предыдущий чат.

Score

Разрешено

Только действия, прямо указанные в prompt и командах этого эксперимента.

Запрещено

Секреты, изменение main/upstream, расширение score и скрытое исправление новых ошибок.

Результат

Артефакты и критерии на третьей странице эксперимента.

Эксперимент 4: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

```
Проведи независимый read-only аудит Open-LLM-VTuber. Не используй и не ищи отчёты предыдущих экспериментов.

Ничего не изменяй. Не читай .env, evidence, .git, .venv, node_modules, логи и backup-файлы. На основании инструкций, документации и кода составь архитектурную карту потока: запуск процесса -> загрузка и валидация конфигурации -> инициализация service context -> LLM/ASR/TTS -> WebSocket -> frontend.

Затем назови ровно три главных риска воспроизведения upstream baseline на Windows. Для каждого укажи категорию, локальное доказательство file:line, минимальную read-only проверку и безопасный способ устранения. Всё недоказанное пометь NOT PROVEN.

В конце выполни git status --short. Не сравнивай результат с первым аудитом.
```

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
# После получения двух независимых отчётов - в отдельной сессии:
# передайте тексты как ОТЧЁТ 1 и ОТЧЁТ 2, затем используйте prompt сравнения из документа.
git status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 4: проверка результата

Что должно получиться

- Получена вторая карта без ссылок на первый отчёт.
- Названы ровно три Windows-риска.
- После отдельного сравнения создана таблица совпадений и расхождений.

Типовые ошибки и действия

- Вторая сессия видит evidence: явно запретить чтение evidence.
- Модель выбирает победителя по уверенности: потребовать локальную команду проверки.
- Отчёты различаются терминами, но не фактами: нормализовать названия компонентов.

Артефакты и доказательства

[] Два независимых отчёта.

[] Таблица расхождений.

[] Реальные команды проверки спорных мест.

Критерий завершения

[] Каждое принятое утверждение доказано локально.

[] Изменений нет.

[] Не осталось неотмеченных предположений.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 4: Независимый аудит Codex | Статус: PASS / FAIL / NOT PROVEN
Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
Source tree: unchanged / <известный diff> | Следующий шаг: Перейти к аудиту допустимых backend-значений текущей версии.

Эксперимент 5. Аудит backend-контрактов

Атомарная цель. Зафиксировать точные допустимые значения LLM, ASR и TTS из Pydantic-схемы версии 1.2.1.

Что мы делаем и зачем

Названия backend являются контрактом. Строка, похожая на имя Python-класса, может не пройти Literal-валидацию. Поэтому значения нельзя угадывать по названию файла или старой инструкции: их берут из схемы установленного commit.

Что должно быть готово

- Архитектурная карта указывает config_manager.
- Никакие настройки пока не меняются.

Где выполнять

Read-only сессия Codex в корне репозитория.

Scope

Разрешено

Только действия, прямо указанные в prompt и командах этого эксперимента.

Запрещено

Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.

Результат

Артефакты и критерии на третьей странице эксперимента.

Эксперимент 5: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Проведи read-only аудит backend-контрактов текущего Open-LLM-VTuber.

Ничего не изменяй. Прочитай только Pydantic-конфигурации LLM, ASR и TTS, default conf template и соответствующие factory-файлы. Покажи точные YAML paths и все Literal-значения для asr_model и tts_model. Отдельно подтверди наличие ollama_llm, openai_compatible_llm, sherpa_onnx_asr и pyttsx3_tts.

Для четырёх выбранных backend укажи: точное значение; YAML path; schema file:line; factory file:line; обязательные дочерние ключи. Не раскрывай содержимое пользовательского conf.yaml. В конце git status -short.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
rg -n "asr_model: Literal|tts_model: Literal|ollama_llm|openai_compatible_llm"
src\open_llm_vtuber\config_manager src\open_llm_vtuber\agent config_templates\conf.default.yaml
git status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 5: проверка результата

Что должно получиться

- ASR содержит sherpa_onnx_asr.
- TTS содержит pyttsx3_tts.
- LLM pool содержит ollama_llm и openai_compatible_llm.
- Пути начинаются с character_config для ASR/TTS и agent_config для LLM.

Типовые ошибки и действия

- Literal error: переписать значение ровно как в схеме.
- Старый conf.yaml другой версии: не чинить шаблон, а классифицировать migration/configuration.
- Комментарий template расходится со схемой: схема является источником истины для валидации.

Артефакты и доказательства

[] Таблица точных значений.

[] YAML paths.

[] Ссылки file:line на schema и factory.

Критерий завершения

[] Ни одно значение не угадано.

[] Пользовательский conf.yaml не опубликован.

[] Source tree неизменен.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 5: Аудит backend-контрактов | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Перейти к чистому upstream baseline до локальной настройки моделей.

Эксперимент 6. Upstream baseline

Атомарная цель. Выполнить `uv sync` и штатный `verbose`-запуск без изменения `source`, `templates` и `dependency versions`.

Что мы делаем и зачем

Baseline отделяет проблемы проекта от последующей настройки. Если сначала изменить `conf.yaml` или код, невозможно доказать, что `upstream` работал в исходном виде. `uv sync` разрешён как штатная материализация уже закреплённых зависимостей; `pyproject.toml` и `uv.lock` редактировать нельзя.

Что должно быть готово

- Рабочая ветка чистая.
- Python 3.10-3.12 доступен через `uv`.
- Порт 12393 свободен или известен процесс, который его занимает.

Где выполнять

Обычный PowerShell в корне Open-LLM-VTuber. Сервер запускается в отдельном окне, чтобы его можно было остановить `Ctrl+C`.

Scope

Разрешено	Только действия, прямо указанные в <code>prompt</code> и командах этого эксперимента.
Запрещено	Секреты, изменение <code>main/upstream</code> , расширение <code>scope</code> и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 6: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

```
Работай как инженер в режиме контролируемого агента.
Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и
изменения, риски и команды проверки.
Сначала покажи план. Не меняй файлы до моего явного подтверждения.
Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
Не переключай и не изменяй main.
Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
Не трогай файлы за пределами указанного score.
Не скрывай ошибки и не подменяй тест словесным утверждением.
После изменения покажи краткий diff, реальный минимальный тест и git status --short.
Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: только воспроизвести upstream baseline. До команд прочитай README.md, CLAUDE.md,
pyproject.toml и quick-start docs. Не меняй source files, templates и dependency versions.

Сначала покажи план и команды. После подтверждения выполни uv sync, затем uv run run_server.py --
verbose. Сохрани реальный вывод, но обезличь пользовательские пути и содержание речи. Классифицируй
каждую ошибку как environment, configuration, dependency или source. Не исправляй ошибку в этом же
score. После проверки останови сервер Ctrl+C и покажи git status --short.
```

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
Set-Location C:\dev\agent-lab\Open-LLM-VTuber
uv sync
uv run run_server.py --verbose
# Остановка: Ctrl+C
git status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажимите Esc и объясните ограничение.

Эксперимент 6: проверка результата

Что должно получиться

- uv sync завершается exit code 0 либо выдаёт сохранённую dependency/environment ошибку.
- Verbose-лог содержит версию Open-LLM-VTuber 1.2.1 и этапы инициализации.
- Локальная страница доступна по адресу из лога.
- После Ctrl+C порт освобождён.

Типовые ошибки и действия

- Pydantic ValidationError: configuration, не dependency.
- Frontend missing: проверить состояние submodule, не изменяя source.
- Port already in use: environment; определить процесс read-only командой.
- Сервер продолжает работать после теста: остановить Ctrl+C и проверить порт.

Артефакты и доказательства

- [] Полный обезличенный uv sync log.
- [] Verbose startup log.
- [] Скриншот локальной страницы без персональных данных.
- [] HTTP/health статус, если endpoint подтверждён документацией.

Критерий завершения

- [] Baseline запущен или ошибка воспроизводима и классифицирована.
- [] Dependency versions не менялись.
- [] Source/templates неизменны.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 6: Upstream baseline | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: После baseline настроить только локальный Ollama VLM в untracked conf.yaml.

Эксперимент 7. Локальный Ollama VLM baseline

Атомарная цель. Подключить qwen3-vl:4b через существующий ollama_llm, изменив только локальный conf.yaml.

Что мы делаем и зачем

Этот эксперимент доказывает локальный text/vision путь без cloud key. Ollama уже предоставляет OpenAI-compatible endpoint на 11434/v1, поэтому отдельный gateway пока не нужен. Изменение ограничивается пользовательским conf.yaml.

Что должно быть готово

- ollama list показывает qwen3-vl:4b.
- ollama serve слушает 127.0.0.1:11434.
- Upstream baseline завершён.

Где выполнять

Codex в корне репозитория; Ollama запущена в отдельном обычном PowerShell.

Scope

Разрешено

Только действия, прямо указанные в prompt и командах этого эксперимента.

Запрещено

Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.

Результат

Артефакты и критерии на третьей странице эксперимента.

Эксперимент 7: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

```
Работай как инженер в режиме контролируемого агента.
Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и
изменения, риски и команды проверки.
Сначала покажи план. Не меняй файлы до моего явного подтверждения.
Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
Не переключай и не изменяй main.
Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
Не трогай файлы за пределами указанного score.
Не скрывай ошибки и не подменяй тест словесным утверждением.
После изменения покажи краткий diff, реальный минимальный тест и git status --short.
Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: изменить только локальный conf.yaml, который не tracked Git. Не меняй source code,
templates и dependencies.

Сначала найди существующие keys
character_config.agent_config.agent_settings.basic_memory_agent.llm_provider и
character_config.agent_config.llm_configs.ollama_llm. Покажи текущие значения без секретов и точечный
YAML plan. После подтверждения установи llm_provider: ollama_llm, base_url: http://127.0.0.1:11434/v1
и model: qwen3-v1:4b. Остальные keys не перестраивай.

После изменения покажи diff только выбранных YAML keys без персонального содержимого, запусти сервер
verbose и предложи text-only и screenshot smoke tests. В конце git status --short.
```

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
ollama serve
# Во втором окне:
ollama list
Set-Location C:\dev\agent-lab\Open-LLM-VTuber
uv run run_server.py --verbose
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 7: проверка результата

Что должно получиться

- Лог инициализации указывает ollama_llm и qwen3-vl:4b.
- Text-only запрос получает осмысленный ответ.
- Screenshot без личных данных описан моделью.
- Нет обращения к cloud API.

Типовые ошибки и действия

- Connection refused 11434: запустить ollama serve вне sandbox.
- Model not found: проверить точное имя ollama list.
- Vision ignored: убедиться, что выбран VLM и payload содержит image по фактическому frontend path.
- conf.yaml tracked: остановиться и проверить .gitignore.

Артефакты и доказательства

[] Три изменённых значения без полного conf.yaml.

[] Text-only ответ.

[] Screenshot и краткий vision-ответ.

[] Обезличенный verbose log.

Критерий завершения

[] Оба smoke test прошли.

[] Cloud key не требовался.

[] Source/templates/dependencies неизменны.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 7: Локальный Ollama VLM baseline | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Настроить локальный русский ASR/TTS baseline.

Эксперимент 8. Русский локальный voice baseline

Атомарная цель. Выбрать sherpa_onnx_asr и pyttsx3_tts, проверить русское распознавание и локальное воспроизведение без cloud API.

Что мы делаем и зачем

Голосовой цикл состоит из независимых частей: захват микрофона, VAD, ASR, LLM и TTS. Один успешный текстовый ответ не доказывает микрофон или звук. Поэтому сохраняются отдельные доказательства распознавания фразы с уникальным числом и слышимого ответа.

Что должно быть готово

- Эксперимент 7 работает.
- Микрофон разрешён браузеру на localhost.
- В Windows установлен хотя бы один русский SAPI voice.

Где выполнять

Обычный PowerShell для списка voices и сервера; браузер для одного voice cycle.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 8: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

```
Работай как инженер в режиме контролируемого агента.
Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и
изменения, риски и команды проверки.
Сначала покажи план. Не меняй файлы до моего явного подтверждения.
Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
Не переключай и не изменяй main.
Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
Не трогай файлы за пределами указанного scope.
Не скрывай ошибки и не подменяй тест словесным утверждением.
После изменения покажи краткий diff, реальный минимальный тест и git status --short.
Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: настроить только local voice baseline в conf.yaml. Не меняй source code, templates и
dependencies. Не отправляй аудио в cloud API.

Сначала найди точные существующие ASR/TTS keys и Pydantic Literal-значения. Покажи план для
character_config.asr_config.asr_model = sherpa_onnx_asr и character_config.tts_config.tts_model =
pyttsx3_tts. Найди pyttsx3_tts.voice_name, если он поддерживается текущей схемой. После подтверждения
измени только эти keys.

Затем выполни PowerShell-совместимый список pyttsx3 voices, запусти verbose server и проведи один
microphone smoke test с фразой «Проверка локального распознавания 4827». В логах не показывай другое
содержание речи. После одного ответа выключи микрофон и проверь отсутствие feedback loop.
```

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
@'
import pyttsx3
engine = pyttsx3.init()
for voice in engine.getProperty("voices"):
    print(voice.id, voice.name, getattr(voice, "languages", []))
'@ | uv run python -

uv run run_server.py --verbose
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами scope или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 8: проверка результата

Что должно получиться

- Список voices напечатан без персональных путей.
- ASR transcript содержит смысл фразы и уникальное число 4827.
- Пользователь слышит локальный TTS-ответ.
- После выключения микрофона новых циклов нет.

Типовые ошибки и действия

- pytsx3 literal_error: использовать pytsx3_tts, а не имя Python-класса.
- TTS object has no attribute: это source/backend contract; не патчить source внутри config-only шага.
- Русский voice отсутствует: установить Windows language/voice отдельно, не добавлять cloud TTS.
- Feedback loop: снизить громкость, включить VAD/echo control и остановить capture после одного цикла.

Артефакты и доказательства

- [] Список voices.
- [] Выбранные ASR/TTS keys.
- [] Transcript только тестовой фразы с 4827.
- [] Ручное подтверждение слышимого ответа.
- [] Наблюдение отсутствия loop.

Критерий завершения

- [] ASR приемлемо распознал русский текст без cloud key.
- [] TTS прозвучал локально.
- [] Нет бесконечного feedback loop.
- [] Source tree неизменен.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 8: Русский локальный voice baseline | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Если эксперименты 1-8 уже выполнялись, пройти контрольную точку 8.5 вместо полного повтора.

Эксперимент 8.5. Контрольная точка продолжения

Атомарная цель. Сопоставить уже выполненные эксперименты 1-8 с критериями и повторить только недостающие доказательства.

Что мы делаем и зачем

Контрольная точка предотвращает две крайности: слепой повтор всей работы и необоснованное предположение, что старый результат соответствует новой методике. Статус «работает» отделяется от статуса «доказано по критерию».

Что должно быть готово

- Собраны доступные логи, скриншоты и Git-история предыдущей работы.
- Никакие старые файлы автоматически не удаляются.

Где выполнять

Read-only Codex в корне Open-LLM-VTuber. Артефакты передаются выборочно и без секретов.

Score

Разрешено

Только действия, прямо указанные в prompt и командах этого эксперимента.

Запрещено

Секреты, изменение main/upstream, расширение score и скрытое исправление новых ошибок.

Результат

Артефакты и критерии на третьей странице эксперимента.

Эксперимент 8.5: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Проведи read-only checkpoint экспериментов 1-8. Не меняй файлы и не исправляй историю.

Для каждого эксперимента создай строку: требование, найденное доказательство, PASS/FAIL/NOT PROVEN, чего не хватает, минимальная команда повтора. Не считай словесное утверждение тестом.

Отдельно проверь три возможных отклонения: менялся ли source ради pytt3x3; были ли evidence committed в репозиторий; был ли PR направлен в upstream вместо собственного fork. Не удаляй и не закрывай ничего без нового score.

В конце выведи только список минимальных повторов и `git status --short`.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
git status --short
git log --oneline -10
git diff --stat
git remote -v
git branch --show-current
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 8.5: проверка результата

Что должно получиться

- Матрица содержит восемь экспериментов.
- Для каждого NOT PROVEN указана одна минимальная проверка.
- Отклонения описаны без автоматического исправления.

Типовые ошибки и действия

- Старое evidence содержит персональные данные: редактировать копию, не оригинал.
- Source менялся: эксперимент может быть функционально успешен, но критерий source unchanged остаётся FAIL.
- PR target неверен: требуется отдельное решение, не silent rewrite history.

Артефакты и доказательства

[] Матрица 1-8.

[] Список недостающих доказательств.

[] Список отклонений и их влияние.

Критерий завершения

[] Не повторяется уже доказанное.

[] Нет скрытых FAIL.

[] Понятна стартовая точка эксперимента 9.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 8.5: Контрольная точка продолжения | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Начать отдельный проект минимального gateway.

Эксперимент 9. Минимальный runtime gateway

Атомарная цель. Создать отдельный loopback-сервис с OpenAI-compatible /v1/chat/completions, Bearer auth и allowlist backend/model.

Что мы делаем и зачем

Codex является инженерным агентом, но не постоянным HTTP endpoint. Gateway заполняет именно этот runtime-разрыв: приложение получает стабильный локальный API, а маршрутизация на Ollama или разрешённый cloud backend остаётся в одном контролируемом сервисе.

Что должно быть готово

- Эксперимент 8.5 завершён.
- Папка C:\dev\agent-lab\local-agent-gateway отсутствует либо её состояние предварительно проверено.
- Ollama отвечает локально.

Где выполнять

Новый sibling-проект C:\dev\agent-lab\local-agent-gateway. Open-LLM-VTuber в этом эксперименте не меняется.

Score

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение score и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 9: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Работай как инженер в режиме контролируемого агента.
 Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и изменения, риски и команды проверки.
 Сначала покажи план. Не меняй файлы до моего явного подтверждения.
 Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
 Не переключай и не изменяй main.
 Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
 Не трогай файлы за пределами указанного scope.
 Не скрывай ошибки и не подменяй тест словесным утверждением.
 После изменения покажи краткий diff, реальный минимальный тест и git status --short.
 Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: создать отдельный минимальный Python gateway в C:\dev\agent-lab\local-agent-gateway.
 Не меняй Open-LLM-VTuber.

Требования: FastAPI; GET /health; POST /v1/chat/completions; bind только 127.0.0.1:8642; Bearer auth для /v1; allowlist backend/model; backend ollama по умолчанию; upstream http://127.0.0.1:11434/v1; нейтральные ошибки; timeout; request_id; не логировать Authorization, полный prompt и полный response. Реальные секреты только в untracked .env, в репозитории только .env.example с placeholders.

Сначала покажи карту проекта, файлы, схему request flow, зависимости, риски и тесты. После подтверждения создай проект. После каждого изменения покажи git diff и минимальный pytest. Не добавляй OpenRouter/OpenAI вызов, пока локальный Ollama path не проверен.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
Set-Location C:\dev\agent-lab
uv init local-agent-gateway
Set-Location .\local-agent-gateway
uv add fastapi "uvicorn[standard]" httpx pydantic-settings
uv add --dev pytest pytest-asyncio
uv run pytest -q
uv run uvicorn app.main:app --host 127.0.0.1 --port 8642
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами scope или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 9: проверка результата

Что должно получиться

- GET /health возвращает 200 без auth и без секретов.
- POST /v1/chat/completions без Bearer возвращает 401.
- Разрешённая модель qwen3-vl:4b проходит к Ollama.
- Запрещённый backend/model отклоняется до сетевого вызова.

Типовые ошибки и действия

- Port 8642 busy: определить процесс, не менять порт молча.
- 401 при правильном клиенте: проверить заголовок без печати token.
- 502/503: проверить Ollama health и timeout.
- Секрет попал в Git: остановиться; не переписывать историю без отдельного решения.

Артефакты и доказательства

- [] Карта gateway-проекта.
- [] OpenAPI schema.
- [] pytest auth/health/allowlist.
- [] Лог bind 127.0.0.1:8642 без prompt.

Критерий завершения

- [] Все тесты имеют exit code 0.
- [] Gateway не слушает 0.0.0.0.
- [] Секреты не tracked.
- [] Open-LLM-VTuber неизменен.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 9: Минимальный runtime gateway | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Подключить Open-LLM-VTuber к gateway только через local conf.yaml.

Эксперимент 10. Интеграция gateway с Open-LLM-VTuber

Атомарная цель. Перенаправить LLM-трафик на 127.0.0.1:8642 только локальной конфигурацией.

Что мы делаем и зачем

Интеграция должна доказать, что приложение воспринимает gateway как обычный OpenAI-compatible provider. Если для этого приходится менять factory или templates, граница совместимости не выдержана и эксперимент останавливается до отдельного adapter-шага.

Что должно быть готово

- Gateway health зелёный.
- Ollama работает.
- Известен локальный gateway token, но он не выводится и не сохраняется в evidence.

Где выполнять

Три процесса в отдельных окнах: Ollama, gateway, Open-LLM-VTuber. Codex меняет только local conf.yaml после подтверждения.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 10: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

```
Работай как инженер в режиме контролируемого агента.
Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и
изменения, риски и команды проверки.
Сначала покажи план. Не меняй файлы до моего явного подтверждения.
Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
Не переключай и не изменяй main.
Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
Не трогай файлы за пределами указанного scope.
Не скрывай ошибки и не подменяй тест словесным утверждением.
После изменения покажи краткий diff, реальный минимальный тест и git status --short.
Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: изменить только локальный untracked conf.yaml Open-LLM-VTuber. Не меняй source,
templates и dependencies.

Сначала найди точные keys openai_compatible_llm и basic_memory_agent.llm_provider. Покажи plan для
llm_provider: openai_compatible_llm, base_url: http://127.0.0.1:8642/v1, model: qwen3-v1:4b и
llm_api_key: <локальный token>. Не показывай реальное значение token ни до, ни после изменения.

После подтверждения измени только эти keys. Запусти verbose server и сделай один text-only smoke test.
Сопоставь request_id приложения и gateway. Покажи git status --short.
```

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
# Окно 1
ollama serve

# Окно 2
Set-Location C:\dev\agent-lab\local-agent-gateway
uv run uvicorn app.main:app --host 127.0.0.1 --port 8642

# Окно 3
Set-Location C:\dev\agent-lab\Open-LLM-VTuber
uv run run_server.py --verbose
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами scope или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 10: проверка результата

Что должно получиться

- VTuber отправляет запрос на :8642, а не напрямую на :11434.
- Gateway логирует request_id и backend/model без prompt.
- Ответ возвращается в UI.
- Source tree VTuber чистый.

Типовые ошибки и действия

- Connection refused 8642: запустить gateway и проверить /health.
- 401: локально сверить token; не печатать его.
- Model denied: добавить модель в allowlist отдельным конфигурационным шагом.
- Требуется source patch: остановиться и зафиксировать contract gap.

Артефакты и доказательства

[] Обезличенные выбранные keys.

[] Gateway request_id.

[] Text-only ответ.

[] git status --short обоих проектов.

Критерий завершения

[] Единый endpoint работает.

[] Cloud не вызывается.

[] VTuber source/templates/dependencies неизменны.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 10: Интеграция gateway с Open-LLM-VTuber | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Проверить vision и voice path через единый gateway.

Эксперимент 11. Экран и голос через gateway

Атомарная цель. Проверить один vision-запрос и один локальный voice cycle с контролем приватности и feedback loop.

Что мы делаем и зачем

Text-only тест не доказывает перенос изображения, а voice baseline не доказывает работу через gateway. Здесь связываются все компоненты: screenshot -> VLM через gateway -> ответ -> локальный TTS, при этом аудио остаётся локальным.

Что должно быть готово

- Эксперимент 10 PASS.
- Подготовлен безопасный screenshot без имён, чатов, token и cookies.
- Динамики на умеренной громкости.

Где выполнять

Те же три окна процессов и браузер localhost. Логи собираются обезличенно.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 11: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Проведи два ограниченных smoke test без изменения файлов.

Тест А: отправь приложенный обезличенный screenshot в Open-LLM-VTuber и задай короткий вопрос о видимом объекте. Докажи по request_id, что payload прошёл через gateway к локальному qwen3-v1:4b.

Тест В: один раз включи microphone capture и попроси меня произнести «Шлюз локального голоса 7319». В логах сохрани только метку [REDACTED SPEECH] и уникальное число 7319. После одного TTS-ответа попроси меня выключить микрофон и 15 секунд наблюдай отсутствие новых циклов.

Не отправляй аудио в cloud API. Не сохраняй исходный audio. Не выводи полные prompt/response. Ничего не изменяй. В конце дай PASS/FAIL отдельно для vision, ASR, gateway, TTS и loop control.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
# Процессы уже запущены. Read-only проверки:
Invoke-RestMethod http://127.0.0.1:8642/health
Get-NetTCPConnection -LocalPort 11434,8642,12393 -State Listen
git status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажимите Esc и объясните ограничение.

Эксперимент 11: проверка результата

Что должно получиться

- Vision-ответ относится к содержимому screenshot.
- Request id связывает VTuber и gateway.
- ASR transcript сохраняет число 7319.
- TTS слышен локально; после выключения микрофона нет нового request_id.

Типовые ошибки и действия

- Screenshot содержит PII: заменить безопасным тестовым изображением.
- Vision payload отсутствует: проверить frontend message type read-only.
- Повторные циклы: немедленно выключить capture и классифицировать VAD/echo control.
- Audio ушёл в cloud: FAIL и остановка эксперимента.

Артефакты и доказательства

- [] Безопасный screenshot.
- [] Request id vision.
- [] Transcript с 7319.
- [] Ручное подтверждение TTS.
- [] 15-секундное наблюдение loop.

Критерий завершения

- [] Vision, ASR, gateway и TTS имеют отдельные доказательства.
- [] Аудио не покидало машину.
- [] Нет feedback loop.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 11: Экран и голос через gateway | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Проверить, нужен ли вообще adapter между VTuber и gateway.

Эксперимент 12. Adapter только при доказанном разрыве

Атомарная цель. Создать минимальный adapter лишь при наличии воспроизводимого несовместимого request/response контракта.

Что мы делаем и зачем

Speculative adapter увеличивает код и скрывает реальную совместимость. Правильный результат эксперимента может быть NOT REQUIRED. Код разрешается только после failing contract test, который показывает конкретное несовпадающее поле.

Что должно быть готово

- Есть sanitized request/response наблюдение эксперимента 11.
- Gateway tests зелёные.

Где выполнять

Только local-agent-gateway. Open-LLM-VTuber остаётся read-only.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 12: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Работай как инженер в режиме контролируемого агента.
 Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и изменения, риски и команды проверки.
 Сначала покажи план. Не меняй файлы до моего явного подтверждения.
 Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
 Не переключай и не изменяй main.
 Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
 Не трогай файлы за пределами указанного score.
 Не скрывай ошибки и не подменяй тест словесным утверждением.
 После изменения покажи краткий diff, реальный минимальный тест и git status --short.
 Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: определить, нужен ли adapter для совместимости Open-LLM-VTuber с gateway.

Сначала read-only сравни фактический sanitized request/response с поддерживаемой схемой gateway. Если формат совместим, не меняй код: зафиксируй NOT REQUIRED и покажи доказательство.

Если несовместимость доказана, сначала добавь ровно один failing contract test. Покажи его реальный FAIL. После нового подтверждения реализуй минимальное преобразование только несовместимых полей, сохрани request_id/status и нейтральные ошибки. Затем покажи diff и PASS того же теста плюс полного pytest. Не добавляй features сверх контракта.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
Set-Location C:\dev\agent-lab\local-agent-gateway
uv run pytest -q
git diff --check
git status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 12: проверка результата

Что должно получиться

- Совместимый формат приводит к NOT REQUIRED без diff.
- При разрыве существует тест, который сначала падает, затем проходит.
- Adapter не переписывает prompt и не раскрывает token.

Типовые ошибки и действия

- Нет sanitized payload: статус NOT PROVEN, код не писать.
- Тест проходит до реализации: он не доказывает разрыв.
- Adapter меняет много полей: сузить score.
- VTuber source предлагается править: остановиться.

Артефакты и доказательства

[] NOT REQUIRED с доказательством либо failing/passing test pair.

[] Минимальный diff adapter.

[] Полный pytest output.

Критерий завершения

[] Нет speculative code.

[] Совместимость доказана тестом.

[] Open-LLM-VTuber source неизменен.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 12: Adapter только при доказанном разрыве | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Перейти к контролируемой инъекции отказов gateway.

Эксперимент 13. Инъекция отказов

Атомарная цель. Доказать безопасное поведение gateway при неверном token, запрещённой модели, недоступном Ollama и timeout.

Что мы делаем и зачем

Happy-path недостаточен для runtime-сервиса. Ошибка upstream не должна превращаться в stack trace, зависание или утечку секретов. Каждый отказ проверяется отдельным автоматическим тестом с ожидаемым HTTP status.

Что должно быть готово

- Gateway unit/contract tests зелёные.
- Ошибки можно симулировать mock transport без остановки пользовательских процессов.

Где выполнять

local-agent-gateway, преимущественно tests/. Реальный Ollama останавливать не требуется, если отказ корректно мокируется.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 13: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Работай как инженер в режиме контролируемого агента.
 Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и изменения, риски и команды проверки.
 Сначала покажи план. Не меняй файлы до моего явного подтверждения.
 Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
 Не переключай и не изменяй main.
 Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
 Не трогай файлы за пределами указанного scope.
 Не скрывай ошибки и не подменяй тест словесным утверждением.
 После изменения покажи краткий diff, реальный минимальный тест и git status --short.
 Если нужен больший score, остановись и запроси новый атомарный шаг.

Добавь только тесты безопасных отказов gateway: отсутствующий/неверный Bearer -> 401; запрещённый backend/model -> отказ до network call; upstream connection failure -> 502 или 503; timeout -> 504 или документированный нейтральный статус. Проверяй отсутствие token, prompt, traceback и локальных путей в body/log capture.

Сначала покажи план тестов и ожидаемую таблицу status. После подтверждения добавь тесты. Меняй production code только если реальный тест обнаружит несоответствие, и только после отдельного подтверждения. Покажи полный pytest output и git diff.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
Set-Location C:\dev\agent-lab\local-agent-gateway
uv run pytest -q
git diff --check
git status --short
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 13: проверка результата

Что должно получиться

- Каждый fault имеет отдельный тест.
- HTTP body нейтрален.
- После симулированного отказа health endpoint продолжает отвечать.

Типовые ошибки и действия

- Тест зависит от реальной сети: заменить mock transport.
- Stack trace попал в body: исправить exception mapping отдельным минимальным diff.
- Token виден в captured log: добавить redaction и regression test.

Артефакты и доказательства

- [] Матрица fault/status.
- [] Полный pytest output.
- [] Проверка отсутствия секретов.

Критерий завершения

- [] Все fault tests PASS.
- [] Процесс остаётся работоспособным.
- [] Нет утечек и недокументированных статусов.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 13: Инъекция отказов | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Описать проверенный операционный runbook.

Эксперимент 14. Операционный runbook

Атомарная цель. Создать подробную инструкцию запуска, остановки, health-проверки, диагностики и отката.

Что мы делаем и зачем

Runbook превращает разовую демонстрацию в воспроизводимую систему. В него попадают только команды, которые реально проверялись в предыдущих экспериментах. Секреты заменяются placeholders, а интерактивные действия явно отделяются от автоматических.

Что должно быть готово

- Известны рабочие порты и команды.
- Fault matrix завершена.
- Есть фактические логи запуска.

Где выполнять

Только README.md проекта local-agent-gateway. Документация Open-LLM-VTuber не меняется.

Scope

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение scope и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 14: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

```
Работай как инженер в режиме контролируемого агента.
Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и
изменения, риски и команды проверки.
Сначала покажи план. Не меняй файлы до моего явного подтверждения.
Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
Не переключай и не изменяй main.
Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
Не трогай файлы за пределами указанного score.
Не скрывай ошибки и не подменяй тест словесным утверждением.
После изменения покажи краткий diff, реальный минимальный тест и git status --short.
Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: дополнить только README.md local-agent-gateway проверенным Windows PowerShell runbook.

Включи prerequisites, структуру окон, порядок Ollama -> gateway -> Open-LLM-VTuber, health checks,
smoke tests, штатную остановку Ctrl+C, проверку освобождения портов, таблицу
симптом/класс/проверка/исправление, rollback local conf.yaml и правила обращения с token. Не вставляй
реальные secrets, абсолютный пользовательский путь и непроверенные команды.

Сначала покажи оглавление и источники каждой команды. После подтверждения измени README, покажи diff,
выполни команды, которые можно безопасно повторить, и полный pytest.
```

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
# Старт
ollama serve
uv run uvicorn app.main:app --host 127.0.0.1 --port 8642
uv run run_server.py --verbose

# Проверки
Invoke-RestMethod http://127.0.0.1:8642/health
Get-NetTCPConnection -LocalPort 11434,8642,12393 -State Listen

# Остановка каждого foreground-процесса: Ctrl+C
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами score или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 14: проверка результата

Что должно получиться

- Новый пользователь понимает, какое окно для какой команды.
- Каждый health check имеет ожидаемый результат.
- Rollback не удаляет данные и не меняет upstream history.

Типовые ошибки и действия

- README содержит непроверенную команду: пометить как NOT VERIFIED или удалить.
- Вставлен реальный token: немедленно заменить placeholder и проверить Git diff.
- Команды зависят от текущей папки: добавить Set-Location перед ними.

Артефакты и доказательства

[] README diff.

[] Повторный pytest.

[] Реальные health outputs.

[] Таблица диагностики.

Критерий завершения

[] Runbook воспроизводит baseline.

[] Нет секретов/PII.

[] Все команды PowerShell-совместимы.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 14: Операционный runbook | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Подготовить намеренную поставку в собственный fork.

Эксперимент 15. Поставка в fork

Атомарная цель. Проверить score, зафиксировать gateway и открыть draft PR в собственном fork без изменения upstream main.

Что мы делаем и зачем

Финальный этап проверяет инженерную дисциплину: commit должен содержать только согласованный gateway и документацию, тесты должны быть воспроизводимы, а публикация выполняется отдельно после показа точного score.

Что должно быть готово

- Все тесты зелёные.
- В Git нет .env, audio, model files и персональных screenshots.
- gh auth status успешен.

Где выполнять

local-agent-gateway. Open-LLM-VTuber публикуется только если там есть отдельно согласованный diff; по умолчанию он остаётся чистым.

Score

Разрешено	Только действия, прямо указанные в prompt и командах этого эксперимента.
Запрещено	Секреты, изменение main/upstream, расширение score и скрытое исправление новых ошибок.
Результат	Артефакты и критерии на третьей странице эксперимента.

Эксперимент 15: готовое выполнение

Откройте Codex из папки, указанной на предыдущей странице. Для независимого аудита обязательно создайте новую сессию. Скопируйте следующий prompt целиком, не вырезая ограничения.

PROMPT ДЛЯ CODEX - КОПИРОВАТЬ ЦЕЛИКОМ

Работай как инженер в режиме контролируемого агента.
 Перед любым изменением покажи карту релевантной части репозитория, одну атомарную цель, файлы чтения и изменения, риски и команды проверки.
 Сначала покажи план. Не меняй файлы до моего явного подтверждения.
 Не используй sudo, force push, git reset --hard, git clean -fd и опасные команды удаления.
 Не переключай и не изменяй main.
 Не читай и не печатай .env, токены, SSH-ключи, cookies и keychain.
 Не трогай файлы за пределами указанного scope.
 Не скрывай ошибки и не подменяй тест словесным утверждением.
 После изменения покажи краткий diff, реальный минимальный тест и git status --short.
 Если нужен больший score, остановись и запроси новый атомарный шаг.

Текущая задача: подготовить поставку local-agent-gateway в собственный GitHub fork.

Сначала read-only покажи branch, remotes, git status --short, diff --stat, список untracked и результаты secret/large-file audit. Затем выполни git diff --check и полный pytest. Сформулируй точный commit scope и draft PR title/body.

Не выполняй commit, push и создание PR до моего отдельного подтверждения. После подтверждения сделай один commit, push только рабочей ветки без force и открой draft PR в мой fork. Покажи URL, target/base и финальный git status --short. Не меняй main и upstream.

Команды PowerShell

Команды ниже нужны либо для ручного выполнения, либо для сверки предложения Codex. Строки после комментария «после подтверждения» не запускаются заранее.

КОМАНДЫ - КОПИРОВАТЬ ПО ПОРЯДКУ

```
git branch --show-current
git remote -v
git status --short
git diff --stat
git diff --check
uv run pytest -q
# commit/push/PR - только после отдельного подтверждения
```

Как действовать при запросе разрешения

1. Прочитайте команду и рабочую папку; сразу сверьте их со score этого эксперимента.
2. Для одной безопасной команды выберите разовое Yes, proceed.
3. Если команда читает секреты, пишет за пределами scope или меняет main - нажмите Esc и объясните ограничение.

Эксперимент 15: проверка результата

Что должно получиться

- Diff содержит только ожидаемые gateway-файлы и README.
- pytest и diff --check PASS.
- Draft PR target - собственный fork и рабочая base branch.
- Нет force push.

Типовые ошибки и действия

- В diff есть .env/audio/model: остановиться и исключить без удаления пользовательских данных.
- PR направлен upstream: не создавать/закрывать автоматически; исправить target до публикации.
- Рабочая ветка main: остановиться.

Артефакты и доказательства

[] Краткий diff.

[] Полный test output.

[] Commit hash.

[] Draft PR URL и target.

[] Финальный status.

Критерий завершения

[] CI/локальные тесты зелёные.

[] Секретов и PII нет.

[] main/upstream не изменены.

ГОТОВАЯ ЗАПИСЬ В ОТЧЁТ

Эксперимент 15: Поставка в fork | Статус: PASS / FAIL / NOT PROVEN
 Доказательство: <минимальный обезличенный вывод> | Не доказано: <ограничение>
 Source tree: unchanged / <известный diff> | Следующий шаг: Выполнить финальный Definition of Done и
 сохранить итоговый отчёт.

Приложение А. Быстрая диагностика Windows

Используйте команды только для локальной диагностики. Они не исправляют проблему автоматически и не должны заменять экспериментальный prompt.

Codex	<code>codex --version; where.exe codex</code>
Sandbox	Минимальная проверка внутри Codex: Выполни Get-Location. Ничего не изменяй.
GitHub	<code>gh auth status</code> ; при <code>invalid credential</code> - <code>gh auth login -h github.com -p https -w</code>
Ollama	<code>ollama --version; ollama list; Test-NetConnection 127.0.0.1 -Port 11434</code>
Gateway	<code>Invoke-RestMethod http://127.0.0.1:8642/health</code>
VTuber	<code>Get-NetTCPConnection -LocalPort 12393 -State Listen</code>

Windows sandbox fallback

Предпочтителен elevated sandbox. Если его setup не работает из-за политики или отсутствия прав, официальный fallback задаётся в %USERPROFILE%\codex\config.toml. Не дублируйте секцию [windows].

```
[windows]
sandbox = "unelevated"
```

Не используйте full access как постоянное лечение. Сначала обновите Codex, повторите setup и проверьте официальный unelevated fallback.

Приложение В. Безопасное обезличивание

Перед сохранением evidence замените персональные данные, но не меняйте технический смысл ошибки. Всегда храните оригинал локально только если он действительно нужен; в Git помещается только безопасная копия после отдельного решения.

Путь	C:\Users\username\... -> %USERPROFILE%\...
Token	Удалить всю строку Token, а не рисовать собственную маску.
Речь	Личную речь -> [REDACTED SPEECH]; уникальное тестовое число можно сохранить.
Prompt	Сохранять только тестовую формулировку без личного контекста.
Screenshot	Использовать специально подготовленное окно без чатов, email, cookies и account UI.
Request log	request_id, backend, model, status, duration; без Authorization и body.

Минимальная проверка текста

```
rg -n "C:\\Users\\|Token:|Authorization:|Bearer " .\evidence
# Любое совпадение проверяется вручную. Не печатайте найденный секрет в отчёт.
```

Приложение С. Prompt сравнения двух аудитов

Запускайте после того, как оба независимых отчёта уже получены. Сравнение не должно переписывать отчёты и выбирать вывод по стилю ответа.

ГОТОВЫЙ PROMPT

Сравни два приложенных независимых аудита Open-LLM-VTuber.

Создай таблицу:

Тема	Отчёт 1	Отчёт 2	Совпадение/расхождение
Команда локальной проверки			Проверенный итог.

Не выбирай утверждение по уверенности формулировки. Каждое расхождение проверь read-only командой по локальным файлам. Если проверить нельзя, поставь NOT PROVEN. Не читай evidence, .env и секреты. Ничего не изменяй. В конце выполни `git status --short`.

Когда аудит считается независимым

- Новая сессия Codex.
- Первый отчёт не передан и запрещено читать evidence.
- Формулировка задачи отличается.
- Финальные расхождения проверяются командами, а не третьим мнением модели.

Приложение D. Definition of Done

- [] Эксперименты 1-8 либо повторены, либо подтверждены контрольной точкой 8.5.
- [] Upstream baseline воспроизводим без изменения source/templates/dependency versions.
- [] Ollama qwen3-vl:4b проходит text и безопасный screenshot smoke tests.
- [] sherpa_onnx_asr распознаёт русскую тестовую фразу без cloud key.
- [] pyttsx3_tts воспроизводит слышимый локальный ответ.
- [] После одного voice cycle нет feedback loop.
- [] Gateway слушает только 127.0.0.1:8642 и требует Bearer для /v1.
- [] Allowlist отклоняет запрещённые backend/model до сетевого вызова.
- [] VTuber использует gateway только через local conf.yaml.
- [] Fault tests покрывают auth, deny, connection failure и timeout.
- [] Runbook содержит только реально проверенные PowerShell-команды.
- [] Секреты, PII, audio и большие модели не tracked.
- [] main и upstream не изменены; публикация только в согласованный fork.

Финальный review prompt

Проведи read-only review итогов практикума по Definition of Done.
 Для каждого пункта верни PASS / FAIL / NOT PROVEN и ссылку на конкретное доказательство.
 Не считай словесное утверждение тестом. Не читай секреты и PII.
 Отдельно проверь source tree VTuber, gateway auth/allowlist, локальность ASR/TTS,
 отсутствие feedback loop и target PR. Ничего не изменяй.

Приложение Е. Источники и границы версии

Команды и ключи Open-LLM-VTuber в этой редакции сверены с локальной версией 1.2.1: README.md, CLAUDE.md, pyproject.toml, config_templates/conf.default.yaml и src/open_llm_vtuber/config_manager/. Перед прохождением на другой версии повторите эксперименты 3-5.

- Codex documentation: <https://developers.openai.com/codex/>
- Codex Windows sandbox: <https://developers.openai.com/codex/windows/>
- OpenAI API reference: <https://platform.openai.com/docs/api-reference>
- Open-LLM-VTuber: <https://github.com/Open-LLM-VTuber/Open-LLM-VTuber>
- Open-LLM-VTuber quick start: <https://open-llm-vtuber.github.io/docs/quick-start>
- Ollama API: <https://docs.ollama.com/api>
- OpenRouter API: <https://openrouter.ai/docs/api-reference/overview>
- FastAPI: <https://fastapi.tiangolo.com/>
- uv: <https://docs.astral.sh/uv/>
- GitHub CLI: <https://cli.github.com/manual/>

Версионность. Если фактический вывод расходится с документом, не подгоняйте систему под текст. Зафиксируйте расхождение, проверьте установленную версию и обновите только соответствующий атомарный шаг.